

Name: Prathamesh Balaji Patil

Roll: 11

Div: TY A(AIML)

Experiment No. 8

Assignment

Naive Bayes Classifier

✓ Campus Message Detective — 3-Class Naive Bayes Classifier

Mini Project Dataset

This version uses a **222-message CSV dataset from Google Drive** instead of embedding the messages directly in the notebook.

Classes: Academic | Placement | Promotional

Algorithm: Multinomial Naive Bayes

Text Vectorization: CountVectorizer

✓ 1. Mount Google Drive

Run the cell below and allow Google Colab to access your Google Drive.

```
from google.colab import drive  
  
drive.mount('/content/drive')
```

Mounted at /content/drive

✓ 2. Load the CSV Dataset from Google Drive

Place `college_messages_222.csv` directly inside **My Drive**.

Expected path:

My Drive/college_messages_222.csv

If you put it inside a folder, change `file_path` accordingly.

```
import pandas as pd

file_path = "/content/drive/MyDrive/Colab Notebooks/college_messages_222.csv"

df = pd.read_csv(file_path)

print("Dataset loaded successfully!")
print("Shape:", df.shape)

df.head()
```

Dataset loaded successfully!
Shape: (222, 2)

	message	label	
0	Mega shopping sale starts today with exciting ...	Promotional	
1	Shop now for exciting deals and special discou...	Promotional	
2	The question bank for the upcoming examination...	Academic	
3	Special festival sale is live with discounts u...	Promotional	
4	Hurry! The biggest sale of the season ends ton...	Promotional	

Next steps:

[Generate code with df](#)

[New interactive sheet](#)

✓ 3. Inspect the Dataset

The CSV should contain two columns:

- `message` — the college message
- `label` — Academic, Placement, or Promotional

```
print("Column names:")
print(df.columns.tolist())

print("\nTotal messages:", len(df))

print("\nClass distribution:")
print(df["label"].value_counts())

print("\nMissing values:")
print(df.isnull().sum())
```

```
Column names:
['message', 'label']

Total messages: 222

Class distribution:
label
Promotional    74
Academic       74
Placement      74
Name: count, dtype: int64

Missing values:
message    0
label      0
dtype: int64
```

✓ 4. Display Sample Messages

```
pd.set_option("display.max_colwidth", 150)

df.head(10)
```

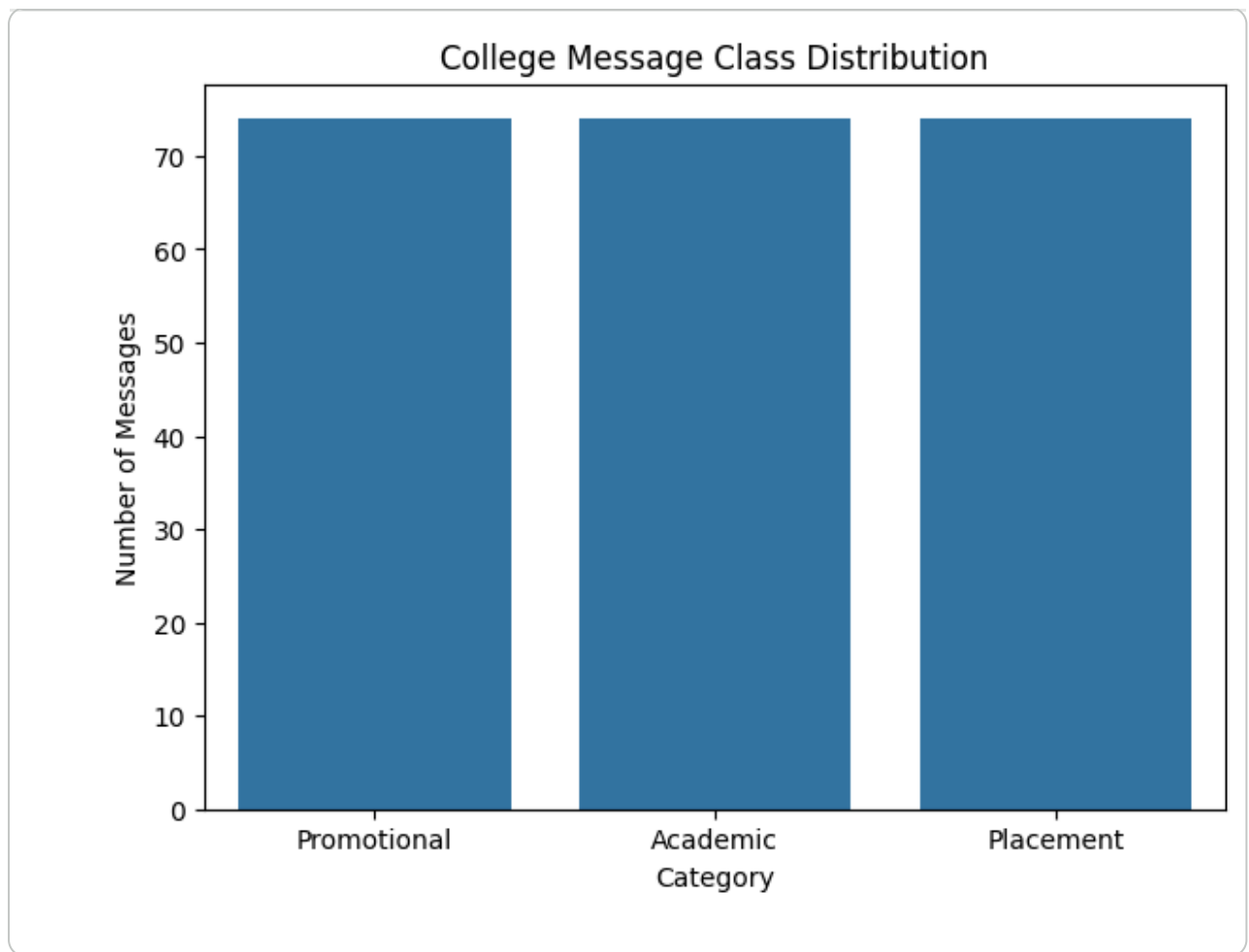
	message	label
0	Mega shopping sale starts today with exciting discounts.	Promotional
1	Shop now for exciting deals and special discounts.	Promotional
2	The question bank for the upcoming examination has been uploaded to the LMS.	Academic
3	Special festival sale is live with discounts up to 80 percent.	Promotional
4	Hurry! The biggest sale of the season ends tonight.	Promotional
5	The placement cell has scheduled a coding practice test for tomorrow.	Placement
6	The company recruitment presentation will be held in the main auditorium.	Placement
7	The department has released the schedule for the practical examinations.	Academic

Next steps: [Generate code with df](#) [New interactive sheet](#)

5. Visualize Class Distribution

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(7, 5))
sns.countplot(data=df, x="label")
plt.title("College Message Class Distribution")
plt.xlabel("Category")
plt.ylabel("Number of Messages")
plt.show()
```



✓ 6. Prepare Features and Labels

```
X = df["message"]
y = df["label"]

print("Features:", X.shape)
print("Labels:", y.shape)
```

```
Features: (222,)
Labels: (222,)
```

✓ 7. Split the Dataset into Training and Testing Sets

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=42,
    stratify=y
)
```

```
print("Training messages:", len(X_train))  
print("Testing messages:", len(X_test))
```

```
Training messages: 166  
Testing messages: 56
```

8. Convert Text into Numerical Features using CountVectorizer

```
from sklearn.feature_extraction.text import CountVectorizer  
  
vectorizer = CountVectorizer(  
    lowercase=True,  
    stop_words="english"  
)  
  
X_train_vectorized = vectorizer.fit_transform(X_train)  
X_test_vectorized = vectorizer.transform(X_test)  
  
print("Training feature matrix:", X_train_vectorized.shape)  
print("Testing feature matrix:", X_test_vectorized.shape)
```

```
Training feature matrix: (166, 374)  
Testing feature matrix: (56, 374)
```

9. Train the Multinomial Naive Bayes Classifier

```
from sklearn.naive_bayes import MultinomialNB  
  
model = MultinomialNB()  
model.fit(X_train_vectorized, y_train)  
  
print("Model training completed successfully!")
```

```
Model training completed successfully!
```

10. Make Predictions

```
y_pred = model.predict(X_test_vectorized)  
  
print("Predictions:")  
print(y_pred)
```

```
Predictions:
['Academic' 'Promotional' 'Academic' 'Placement' 'Promotional'
 'Promotional' 'Placement' 'Placement' 'Placement' 'Academic' 'Academic'
 'Promotional' 'Academic' 'Academic' 'Placement' 'Academic' 'Placement'
 'Placement' 'Promotional' 'Promotional' 'Promotional' 'Promotional'
 'Academic' 'Promotional' 'Placement' 'Academic' 'Academic' 'Promotional'
 'Placement' 'Placement' 'Promotional' 'Academic' 'Placement' 'Placement'
 'Placement' 'Promotional' 'Academic' 'Academic' 'Academic' 'Placement'
 'Promotional' 'Placement' 'Placement' 'Placement' 'Promotional'
 'Academic' 'Placement' 'Placement' 'Promotional' 'Promotional' 'Academic'
 'Academic' 'Promotional' 'Promotional' 'Promotional' 'Placement']
```

✓ 11. Evaluate the Model

```
from sklearn.metrics import accuracy_score, classification_report

accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", round(accuracy * 100, 2), "%")

print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

Accuracy: 98.21 %

Classification Report:

	precision	recall	f1-score	support
Academic	1.00	0.94	0.97	18
Placement	0.95	1.00	0.97	19
Promotional	1.00	1.00	1.00	19
accuracy			0.98	56
macro avg	0.98	0.98	0.98	56
weighted avg	0.98	0.98	0.98	56

✓ 12. Confusion Matrix

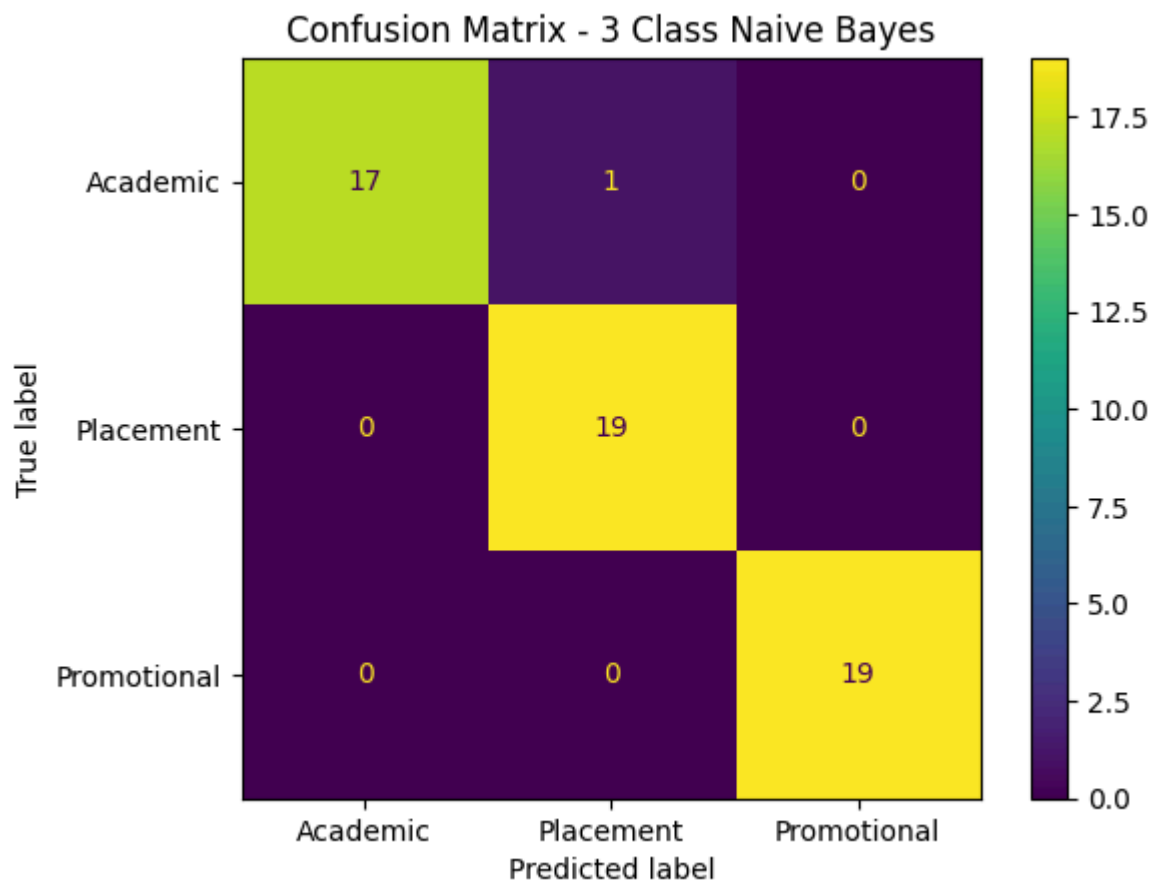
```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

cm = confusion_matrix(y_test, y_pred, labels=model.classes_)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=model.classes_
)

disp.plot()
plt.title("Confusion Matrix - 3 Class Naive Bayes")
```

```
plt.show()
```



✓ 13. Test the Classifier with New Messages

```
test_messages = [
    "The DBMS practical examination will be conducted tomorrow in Lab 2.",
    "The placement cell has announced a campus recruitment drive for eligibl",
    "Get 50 percent discount on selected products. Shop now and claim your o
]
```

```
test_vectors = vectorizer.transform(test_messages)
predictions = model.predict(test_vectors)
```

```
for message, prediction in zip(test_messages, predictions):
    print("Message:", message)
    print("Predicted Category:", prediction)
    print("-" * 80)
```

```
Message: The DBMS practical examination will be conducted tomorrow in Lab 2.
Predicted Category: Academic
```

```
-----
Message: The placement cell has announced a campus recruitment drive for elig
Predicted Category: Placement
```

```
-----
Message: Get 50 percent discount on selected products. Shop now and claim you
```

Predicted Category: Promotional

✓ 14. Interactive Message Classification

```
message = input("Enter a college message: ")

message_vector = vectorizer.transform([message])

prediction = model.predict(message_vector)[0]
probabilities = model.predict_proba(message_vector)[0]

print("\nPredicted Category:", prediction)

print("\nPrediction Probabilities:")
for class_name, probability in zip(model.classes_, probabilities):
    print(f"{class_name}: {probability * 100:.2f}%")
```

Enter a college message: Mega shopping sale starts today with exciting discou

Predicted Category: Promotional

Prediction Probabilities:

Academic: 0.01%

Placement: 0.01%

Promotional: 99.99%

✓ 15. Save the Trained Model and Vectorizer

These files will be used by the Flask application (`app2.py`).

```
import joblib

joblib.dump(model, "naive_bayes_model.pkl")
joblib.dump(vectorizer, "vectorizer.pkl")

print("naive_bayes_model.pkl saved successfully!")
print("vectorizer.pkl saved successfully!")
```

naive_bayes_model.pkl saved successfully!
vectorizer.pkl saved successfully!

✓ 16. Download the Model Files

```
from google.colab import files

files.download("naive_bayes_model.pkl")
files.download("vectorizer.pkl")
```